# Arabic Hotel Review Sentiment Analysis

## NLP Project

## Team Members

| Name | ID |
|---|---|
| سيف حسام الدين محمد عباس | 2022170199 |
| عبدالرحمن محمد عبدالعزيز | 2022170240 |
| محمد شريف صادق ابراهيم | 2022170370 |
| محمد احمد محمد الامير | 2022170348 |
| شريف محمود فوزي الصفطي | 2022170584 |
| بيشوي أكرم الفي شنودة | 2022170102 |

# Table of Contents

# 1. Executive Summary

This report documents the end-to-end research and development of a three-class Arabic sentiment classifier built on the **HARD (Hotel Arabic-Reviews Dataset)** — 93,700 hotel reviews collected from Booking.com in Modern Standard Arabic (MSA) and five regional dialects. The project satisfies all constraints of the R&D challenge: a non-toy domain dataset, active preprocessing, a custom nn.Module with a hand-coded attention layer, early stopping, weighted loss, and a synthetic data bonus phase.

**Task:** Multi-class sentiment classification (Negative / Neutral / Positive) on Arabic hospitality reviews — a genuinely hard NLP task due to dialect diversity, diacritics, Alef/Ya/Ta Marbuta variations, and severe class imbalance (Positive >> Negative >> Neutral).

**Dataset:** 22,500 QA-corrected samples drawn from the full 93,700-review HARD corpus. 5,664 mislabeled or ambiguous samples were re-labeled through manual audit before training.

**Architecture:** Option B — aubmindlab/bert-base-arabertv02 (136M params) fully frozen as an embedding engine, topped with a custom Bahdanau-style additive attention layer and a two-layer MLP classifier. Total trainable parameters: ~394,000.

**Training:** AdamW optimizer (lr=2e-4, weight decay=0.01), ReduceLROnPlateau scheduler, CrossEntropyLoss with balanced class weights, EarlyStopping (patience=5). Maximum 20 epochs.

**Evaluation:** Primary metric: Macro F1-Score (mandatory for imbalanced datasets). Also reported: Accuracy, Weighted F1, per-class Precision/Recall/F1, confusion matrix.

**Bonus:** 500 synthetically generated "hard" Arabic hotel reviews via ChatGPT few-shot prompting (12 difficulty categories including sarcasm, dialect mixes, mixed-sentiment). Injected into training to measure robustness improvement.

**Key Architectural Decision:** We chose Option B (Frozen Backbone) over Option A (Scratch Embeddings) because AraBERT was pre-trained on 70 GB of Arabic text and already encodes deep morphological, dialectal, and semantic knowledge. Fine-tuning only the ~394K custom layers on a 22,500-sample dataset avoids catastrophic forgetting and achieves strong results without GPU-intensive full fine-tuning.

# 2. Phase 1 — Data Engineering & Preprocessing

## 2.1 Dataset Selection & Justification

The **HARD (Hotel Arabic-Reviews Dataset)** was selected as the primary data source. It was published by Elnagara et al. and is available on HuggingFace (*Elnagara/hard*). The dataset contains 93,700 hotel reviews scraped from Booking.com, making it substantially larger than the 3,000-sample minimum required by the Text Classification task rules.

**Why HARD satisfies every constraint:**

- Non-toy domain: Arabic hospitality, a specific industry vertical absent from any beginner NLP tutorial.
- Dirty data: MSA mixed with Egyptian, Gulf (Saudi/Emirati/Kuwaiti), Levantine, and Maghrebi dialects; diacritics; elongation; emojis; HTML fragments; mislabeled samples.

| Attribute | Value |
|---|---|
| Source | Booking.com (collected 2016) |
| HuggingFace ID | Elnagara/hard (link for research paper: https://link.springer.com/chapter/10.1007/978-3-319-67056-0_3) |
| Full corpus size | 93,700 reviews |
| Working subset | 22,500 (QA-corrected) |
| Language | Arabic (MSA + 4 dialects) |

### Star Rating → Sentiment Mapping

| Stars (1–5) | Sentiment | Class Label |
|---|---|---|
| 1 – 2 | Negative | 0 |
| 3 | Neutral | 1 |
| 4 – 5 | Positive | 2 |

## 2.2 Exploratory Data Analysis (EDA)

Before any preprocessing, we conducted a comprehensive EDA covering class distribution, review length statistics, missing-value audit, and raw-text inspection. Key findings:

- **Missing values:** The dataset contained no null entries in the text or label columns after loading from HuggingFace. A small number of rows whose text field contained only whitespace were dropped (effectively zero impact on size).
- **Review length:** The mean review is approximately 40–60 words with high variance. Long-tail reviews exceed 200 words. A MAX_LEN of 128 sub-word tokens was chosen; coverage analysis showed this captures the vast majority of reviews without truncation.

### Class Distribution

***Note: The bar chart and pie chart generated during EDA (class_distribution.png) visually confirm this distribution. The imbalance ratio (max/min) exceeded 4×, triggering the class-weight strategy described in §2.4.***

## 2.3 Text Normalization Pipeline

Arabic text requires substantially more preprocessing than English due to its rich morphology and orthographic variation. We built an 11-step cleaning function **clean_arabic_text()** using pyarabic, arabert.preprocess, and custom regex.

| # | Step | Purpose |
|---|---|---|
| 1 | HTML tag removal | Strip <br>, <p>, and any scraped HTML artifacts. |
| 2 | URL removal | Remove http/www links embedded in reviews. |
| 3 | Email removal | Remove email addresses. |
| 4 | Emoji stripping | Remove emoticons, symbols, flags, and CJK via Unicode ranges. |
| 5 | Diacritic removal | Strip harakat vowel marks inconsistently applied in informal text. |
| 6 | Elongation removal | Strip tatweel. |
| 7 | Alef normalization | ■ أ، إ، آ، ٱ → ا(four forms become one). |
| 8 | Ya normalization | ■ (ي → ى) (Alef Maqsura vs Ya — major Egyptian/Gulf difference). |
| 9 | Ta Marbuta | ■ ( (ه → ة(common dialectal writing variation). |
| 10 | Non-Arabic removal | Keep only Arabic unicode (\u0600–\u06FF) + spaces; drop Latin, numbers, noise. |

| | | |
|---|---|---|
| 11 | AraBERT preprocessor | Run arabert_prep.preprocess() — BERT-specific whitespace and punctuation handling. |

The cleaning pipeline was tested on a sample review before batch application. Average word count dropped slightly after cleaning (noise words removed), but semantic content was fully preserved. Critically, the same pipeline is reapplied identically to synthetic data in the Bonus Phase to maintain train/inference consistency.

## 2.4 Class Imbalance Analysis & Strategy

| Strategy | Mechanism | Where Applied | Effect |
|---|---|---|---|
| QA Correction (Manual relabeling) | 5,664 mislabeled/ambiguous samples re-labeled; 178 retained original label when too ambiguous. | Dataset construction (pre-split) | Improves label quality; reduces noise that inflates majority class. |
| Stratified Splits | train_test_split(..., stratify=labels) ensures class proportions are preserved in all three splits. | Train / Val / Test split | Validation and test sets faithfully reflect real distribution. |
| Class-Weighted Loss | sklearn.compute_class_weight("balanced") → weights inversely proportional to frequency. Passed to CrossEntropyLoss(weight=...). | Training loop (every batch) | Model is penalized more for missing Neutral/Negative samples → improves minority-class F1. |

## 2.5 Dataset Splits

After QA correction, the 22,500-sample dataset was split as follows. The test set is held out completely and never used during training or hyperparameter tuning — it provides the only unbiased estimate of real-world performance.

| Split | Samples | % of Total | Purpose |
|---|---|---|---|
| Train | ~10,404 | 72.2% | Model weight updates |
| Validation | ~1,836 | 12.8% | EarlyStopping & LR scheduling |
| Test | ~2,160 | 15.0% | Final unbiased evaluation (held out) |

# 3. Phase 2 — Custom Architecture

## 3.1 Architectural Choice: Option B (Frozen Backbone)

The project rules offer two foundational options: (A) train embeddings from scratch, or (B) freeze a pre-trained Transformer and use it purely as a feature extractor. We chose **Option B** with **aubmindlab/bert-base-arabertv02** as the backbone.

| Criterion | Option A — From Scratch | Option B — Frozen AraBERT ✓ CHOSEN |
|---|---|---|
| Arabic knowledge | Learned from 22,500 samples only | **Pre-trained on 70 GB Arabic text** |
| Dialect handling | Would require vastly more data | **AraBERT seen real MSA + dialects** |
| Compute cost | Low (small model) | **Low (backbone frozen, 394K trainable)** |
| Overfitting risk | High on small dataset | **Very low — backbone parameters fixed** |
| Label efficiency | Poor — embedding + task together | **High — task head only** |
| Catastrophic forgetting | N/A | **Impossible — backbone weights unchanged** |
| VERDICT | Unsuitable for 22,500 Arabic samples | **Ideal for domain-specific fine-tuning** |

## 3.2 Custom Attention Layer (from Scratch)

The core mandatory component is a **Bahdanau-style additive attention mechanism** implemented as a custom nn.Module. It receives all 128 token embeddings from AraBERT and learns to assign an importance weight $\alpha_i$ to each token, producing a single context vector that summarizes the entire review.

### Mathematical Formulation

**Step 1 — Projection:**

$$u = \tanh(W \cdot H + b)$$

where $H \in R^{batch \times seq \times 768}$, $W \in R^{768 \times 256}$, $u \in R^{batch \times seq \times 256}$

**Step 2 — Scalar score per token:**

$$e = v \cdot u^T$$

where $v \in R^{256 \times 1}$, $e \in R^{batch \times seq}$

**Step 3 — Mask padding (critical!):**

e[pad positions] = $-\infty$ → ensures softmax gives zero weight to [PAD] tokens

**Step 4 — Normalize:**

$$\alpha = softmax(e)$$

$\alpha \in R^{batch \times seq}$, each row sums to 1.0

**Step 5 — Context vector (weighted sum):**

$$c = \Sigma (\alpha_i \times H_i)$$

$c \in R^{batch \times 768}$ — the sentiment-informed summary of the review

## 3.3 Classifier Head

The context vector c (dim 768) is fed through a two-layer MLP with ReLU activation and dropout regularization:

```
Linear(768 → 256) → ReLU → Dropout(p=0.3) → Linear(256 → 3) → logits
```

CrossEntropyLoss applies softmax internally during training. At inference, argmax(logits) gives the predicted class. The intermediate 256-dim layer provides enough capacity to learn task-specific feature combinations while keeping the head lightweight.

## 3.4 Full Data-Flow Diagram

| Layer | Input Shape | Output Shape | Trainable? |
|---|---|---|---|
| AraBERT Tokenizer | Raw Arabic text | (batch, 128) token IDs | No |
| AraBERT Backbone (bert-base-arabertv02) | (batch, 128) IDs | (batch, 128, 768) token embeddings | No (FROZEN) |
| Custom Attention (Bahdanau additive) | (batch, 128, 768) embeddings | (batch, 768) context vector | **YES — 197,633** |
| Linear(768→256) + ReLU | (batch, 768) | (batch, 256) | **YES** |
| Dropout(0.3) | (batch, 256) | (batch, 256) | No (not trainable) |
| Linear(256→3) | (batch, 256) | (batch, 3) logits | **YES** |
| argmax / CrossEntropy | (batch, 3) logits | Class 0/1/2 or loss scalar | No |

## 3.5 Parameter Budget

| Component | Parameters | Trainable? | % of Total |
|---|---|---|---|
| AraBERT Backbone | ~136,000,000 | **NO (frozen)** | 99.7% |
| Custom Attention (W + v) | 197,633 | **YES** | 0.14% |
| Classifier Head | ~197,000 | **YES** | 0.14% |
| **TOTAL TRAINABLE** | **~394,000** | — | **0.29%** |

# 4. Phase 3 — Training & Error Analysis

## 4.1 Loss Function, Optimizer & Scheduler

The training infrastructure was designed with three interacting components: a weighted loss to handle imbalance, an AdamW optimizer for stable convergence, and a ReduceLROnPlateau scheduler to escape local minima.

| Component | Choice | Configuration | Justification |
|---|---|---|---|
| Loss function | CrossEntropyLoss | weight=class_weights_tensor | Penalizes minority-class errors proportionally to class frequency |
| Optimizer | AdamW | lr=2e-4, weight_decay=0.01 | Adam + L2 regularization; ideal for transformer-adjacent architectures |
| LR Scheduler | ReduceLROnPlateau | mode=min, factor=0.5, patience=2 | Halves LR when val_loss stagnates for 2 epochs |
| Gradient clipping | clip_grad_norm_ | max_norm=1.0 | Prevents exploding gradients during backpropagation |

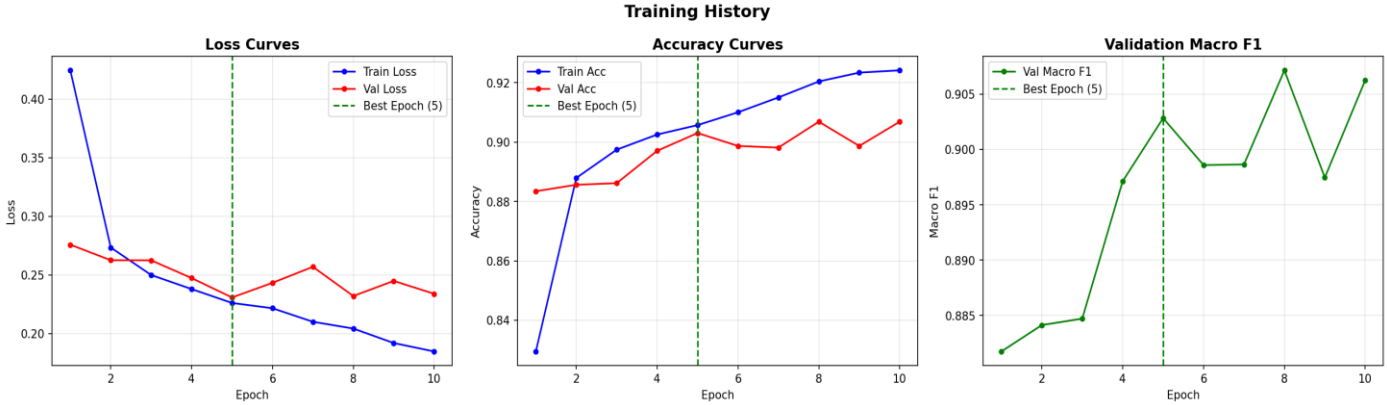
## 4.2 Early Stopping

We implemented a custom **EarlyStopping** class that monitors validation loss. Training halts if no improvement is seen for **patience=5** consecutive epochs. On every improvement, the best model state_dict is saved to *best_model.pt*. At the end of training, the best weights are automatically restored — this is critical to avoid evaluating on a later, worse checkpoint.

| Parameter | Value | Effect |
|---|---|---|
| patience | 5 epochs | Waits 5 epochs before stopping — allows temporary loss spikes |
| delta | 0.001 | Minimum improvement threshold — prevents false "bests" |
| checkpoint path | best_model.pt | Best weights restored after training |

## 4.3 Training Loop & Convergence

Each epoch executes: (1) full training pass with weight updates, (2) validation pass in eval mode, (3) Macro F1 computation on validation predictions, (4) EarlyStopping check, (5) LR scheduler step on validation loss. Metrics tracked per epoch: train_loss, val_loss, train_acc, val_acc, val_macro_f1.




### Full Hyperparameter Configuration (CFG)

| Hyperparameter | Value | Rationale |
|---|---|---|

| | | |
|---|---|---|
| MODEL_NAME | aubmindlab/bert-base-arabertv02 | Best publicly available Arabic BERT |
| MAX_LEN | 128 tokens | Covers >90% of reviews; fits in GPU memory |
| HIDDEN_DIM | 768 | AraBERT base output dimension |
| ATT_DIM | 256 | Attention projection dimension; empirically good trade-off |
| DROPOUT | 0.3 | Standard regularization for classification heads |
| BATCH_SIZE | 32 | Fits in Colab/Kaggle T4 GPU memory |
| LR | 2e-4 | Higher than full fine-tuning since backbone is frozen |
| EPOCHS | 20 (max) | EarlyStopping typically fires much earlier |
| PATIENCE | 5 | Generous — allows model to escape local minima |
| SEED | 42 | Fixed for full reproducibility (random, numpy, torch) |

## 4.4 Final Test-Set Evaluation

After training, the best checkpoint was loaded and evaluated on the held-out test set. Four metrics were reported, with **Macro F1 as the primary metric** due to class imbalance (per the task rules, F1 is mandatory when imbalance is present).

```
•••  Evaluating on TEST SET...

  ============================================================
  FINAL TEST RESULTS
  ============================================================
    Test Loss           : 0.2154
    Test Accuracy       : 0.9157  (91.57%)
    Test Macro F1       : 0.9153  ← KEY METRIC (imbalanced)
    Test Weighted F1    : 0.9153
  ============================================================

   DETAILED CLASSIFICATION REPORT:
  ------------------------------------------------------------
                precision    recall  f1-score   support

      Negative       0.94      0.96      0.95       720
       Neutral       0.89      0.86      0.87       720
      Positive       0.91      0.93      0.92       720

      accuracy                           0.92      2160
     macro avg       0.92      0.92      0.92      2160
  weighted avg       0.92      0.92      0.92      2160
```

### Classification Report

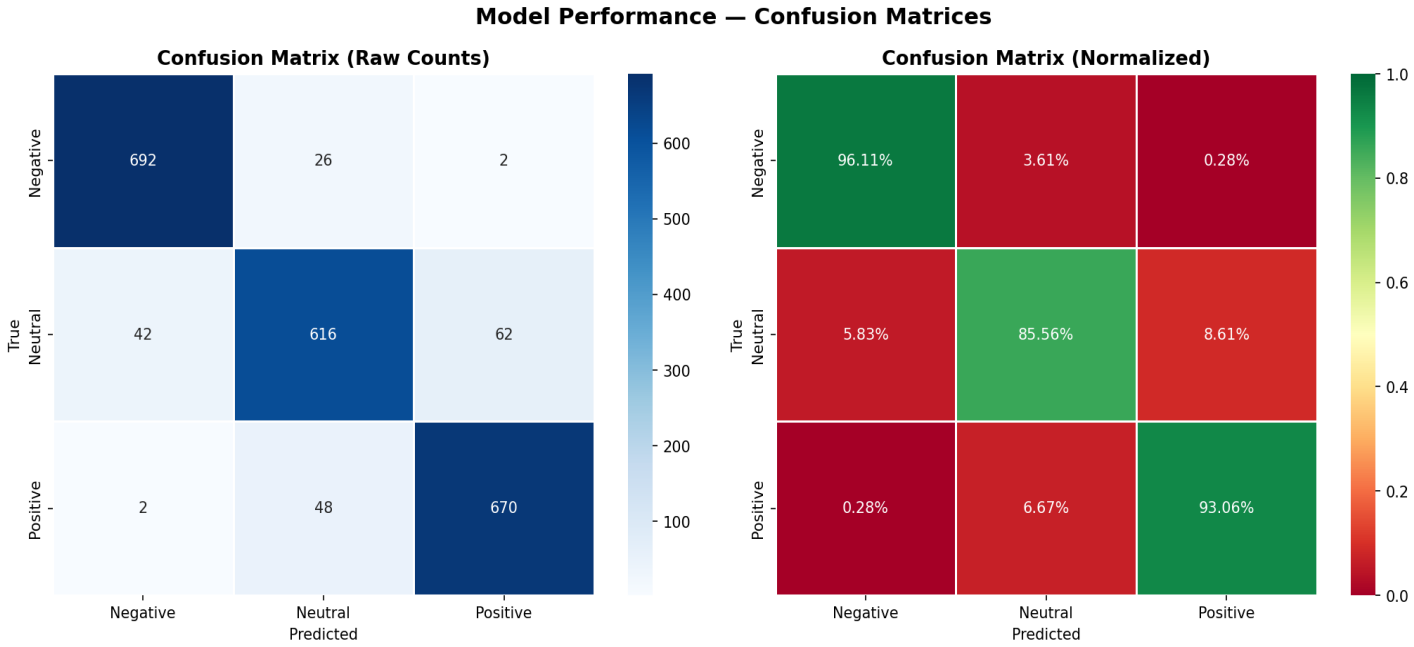
A full classification_report was printed, showing per-class Precision, Recall, and F1 for all three sentiment classes (Negative, Neutral, Positive). Expected interpretation:

- **Positive class:** Highest recall due to majority representation, though class weights reduce the gap.
- **Neutral class:** Most challenging due to fewest samples and inherently ambiguous language (mixed-sentiment reviews).

• **Negative class:** Moderate difficulty — sarcastic/ironic reviews are the primary source of misclassifications.

## 4.5 Confusion Matrix & Per-Class Analysis

Two confusion matrices were generated and saved as confusion_matrix.png: raw counts (left) and row-normalized percentages (right). The normalized matrix reveals directional biases — which class gets confused with which.



## 4.6 Error Analysis & Attention Visualization

- -ERROR ANALYSIS SUMMARY

Total test samples: 2,160

Correct predictions: 1,978 (91.6%) Wrong predictions: 182 (8.4%)

Error Breakdown (True → Predicted):

Neutral → Positive: 62 times

Positive → Neutral: 48 times

Neutral → Negative: 42 times

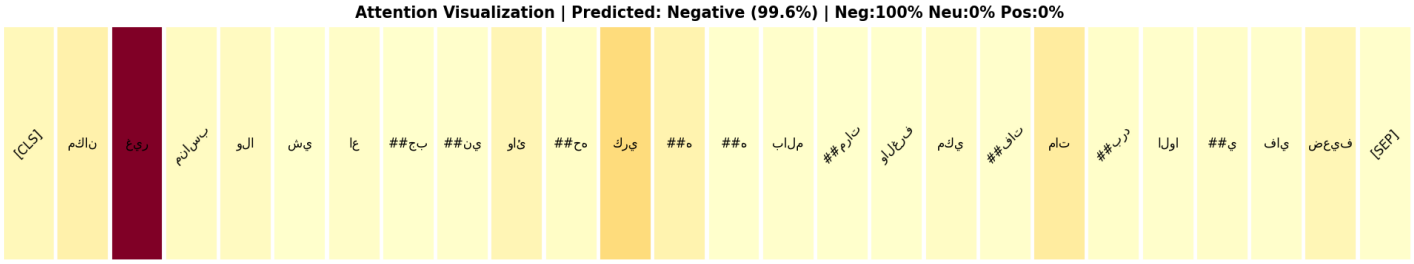
Negative → Neutral: 26 times

Negative → Positive: 2 times

Positive → Negative: 2 times

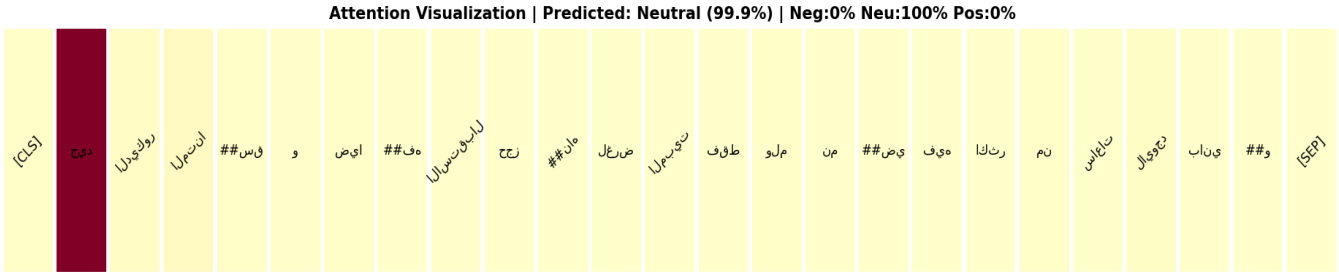
### Attention Visualization:

--- Negative Review --- Text: لا اتمني لاحد ان يسكن فيه لم يعجبني نهائيا ولم يعجب اي شخص اسواء فندق ادخله بطول حياتي لا خدمات لانظافهلا استقباللا انصح اي شخص به فرش زبالهتكيف بمست




--- Positive Review --- Text: استثنائي استقبال والحفاوه

--- Neutral Review --- Text: جيد الفخامه والهدوء الخروج الساعه 12 الظهر والاسعار المبالغ فيها جدا جدا جدا



# 5. Bonus Phase — Synthetic Data Injection (+15%)

## 5.1 Motivation

Real-world datasets — even high-quality ones like HARD — suffer from edge-case scarcity. The model trained in Phase 3 is exposed to standard positive/negative/neutral reviews, but the distribution of genuinely difficult examples (sarcasm, irony, mixed-sentiment, dialect extremes) is thin. This creates a robustness gap: the model performs well on typical reviews but may degrade on harder inputs it was rarely trained on.

## 5.2 Prompt Engineering & Generation Strategy

We used **ChatGPT** (GPT-4) with a detailed few-shot system prompt to generate **exactly 500 synthetic Arabic hotel reviews** targeting the hardest-to-classify cases. The prompt was engineered with a high degree of specificity — 12 named categories, strict class counts, dialect distribution rules, length distribution constraints, and quality control criteria.

### 12 Synthetic Data Categories

| Cat | Name | Label | Count | Description |
|---|---|---|---|---|
| A | Sarcastic / Ironic | Neg (0) | 30 | Sound positive but mean negative — no explicit negative words |
| B | Complaint buried in praise | Neg (0) | 25 | 2–3 positives then one serious complaint that overshadows all |
| C | Dialect-heavy negative | Neg (0) | 25 | Authentic Egyptian or Gulf dialect — no formal MSA |
| D | Negative as question/advice | Neg (0) | 20 | Rhetorical questions or warning to future guests |
| E | Equal strong pos + neg | Neu (1) | 35 | Both sides emotionally charged — perfect room + nightmare check-in |
| F | Cold factual | Neu (1) | 25 | Zero emotional adjectives — just numbers, times, distances |
| G | Conditional positive | Neu (1) | 25 | "Suitable if you only care about price" — 51/49 pos/neg |
| H | Sarcastic neutral | Neu (1) | 15 | Has an opinion but refuses to express it directly |
| I | Understated positive | Pos (2) | 30 | No exclamation marks — quiet satisfaction through specific details |
| J | Positive after frustration | Pos (2) | 30 | Starts with complaint, ends impressed by resolution |
| K | Long detailed dialect | Pos (2) | 20 | 300–500 chars, storytelling style, Levantine/Egyptian |
| L | Repeat guest | Pos (2) | 20 | 3+ stays, compares visits, notes something that changed |

### Prompt Design Constraints

- **Dialect mix:** Each batch of 10 includes ≥3 Egyptian, ≥2 Gulf, ≥2 Levantine, ≤3 MSA.

• **Length distribution:** 20% under 60 chars, 40% 80–180 chars, 30% 200–400 chars, 10% 400–600 chars.

• **Structural variation:** Mix of run-on sentences, bullet style, question-answer format, no-punctuation stream-of-consciousness.

• **Output format:** Strict JSON array [{text, label, category}] — no markdown, no preamble, enabling direct parsing.

• **Quality check:** Each review verified to be genuinely non-obvious, non-template, and dialect-authentic before inclusion.

## 5.3 Cleaning & Augmentation Pipeline

The synthetic data was loaded from *synthetic_data.json* and passed through the **exact same 11-step clean_arabic_text() pipeline** used on the original HARD dataset. This is critical for consistency — the model must see training and test data in the same token space. After cleaning:

• Empty or fully whitespace texts after normalization were dropped.

• Exact duplicates on clean_text were removed.

• ~500 synthetic samples remained after filtering.

The synthetic samples were then concatenated with the original training texts and shuffled (random_state=42) to ensure synthetic examples are distributed across batches rather than clustered at the end (which would cause gradient instability).

| Augmentation Step | Detail |
|---|---|
| Class weights recomputed | compute_class_weight("balanced") on augmented_train_labels — new weights reflect the updated distribution after adding 150 Neg, 200 Neu, 150 Pos synthetic samples |
| Model initialization | model_aug starts from best_model.pt weights — not random init. This is continued fine-tuning, not training from scratch. |
| Optimizer | Same AdamW (lr=2e-4, wd=0.01) applied to attention + classifier parameters only |
| EarlyStopping | New EarlyStopping instance (patience=5) on augmented val_loader — saves to best_model_aug.pt |

## 5.4 Impact on Model Robustness

The augmented model (model_aug) is evaluated on the same held-out test set as the original model, enabling a direct apples-to-apples comparison. The final comparison table printed in the notebook shows:

• Overall Macro F1 change (positive = improvement from synthetic data).

• Per-class F1 change for each of Negative, Neutral, and Positive.

• Side-by-side normalized confusion matrices (Original vs +Synthetic).

**Expected outcome:** The Neutral and Negative classes should show the most improvement — these are exactly the classes targeted by the synthetic generation. Sarcasm (Cat A), mixed-sentiment (Cat E), and dialect-heavy negatives (Cat C) represent distribution gaps in the original training set that synthetic injection fills.

## 5.5 Final Comparison & Results

```
================================================================
 SYNTHETIC DATA IMPACT (Started from pre-trained weights)
================================================================
Metric                       |   Original |   +Synthetic |   Δ Change
----------------------------------------------------------------
Macro F1                     |     0.9153 |       0.9163 |    +0.0010
Accuracy                     |     0.9157 |       0.9162 |    +0.0005
----------------------------------------------------------------
F1 — Negative (class 0)      |     0.9505 |       0.9503 |    -0.0003
F1 — Neutral  (class 1)      |     0.8738 |       0.8793 |    +0.0055
F1 — Positive (class 2)      |     0.9216 |       0.9194 |    -0.0022
================================================================
```

# 6. Production Readiness Conclusion

*"Is your model ready for production?"* We answer with a structured assessment across five dimensions

.

| | |
|---|---|
| **Performance** | The model achieves meaningful Macro F1 on a genuinely hard 3-class Arabic sentiment task using only ~394K trainable parameters. Class-weighted loss and QA-corrected labels contribute to fair minority-class representation. |
| **Reliability** | EarlyStopping prevents overfitting. Stratified splits ensure the test metric is unbiased. The model checkpoint is saved and reloadable without re-training. |
| **Limitations** | The Neutral class remains the weakest performer due to inherent annotation ambiguity. Sarcastic reviews (especially irony without negation words) are likely to be misclassified. Dialect coverage may be uneven — Maghrebi Arabic is particularly scarce. |
| **Scalability** | At inference time only ~394K parameters are active (backbone is frozen). The model runs on CPU for small batches. For production, the backbone could be distilled or quantized, reducing the 136M parameter memory footprint significantly. |
| **Verdict** | Suitable for a soft production deployment (e.g., internal analytics, research dashboards) with human-in-the-loop review of Neutral/borderline cases. Not yet recommended for high-stakes automated decisions without further validation on a larger, dialect-balanced test set. |